

АННОТАЦИЯ

Тема выпускной квалификационной работы — Веб-приложение по подготовке к техническим собеседованиям на основе стартапа ViewTrain.

ViewTrain — технологический стартап в сфере EdTech, представляющий собой сервис симуляции технических собеседований с AI-интервьюером на базе GigaChat. Целевая аудитория продукта — IT-специалисты начинающего уровня, готовящиеся к собеседованиям. Его серверная часть разработана в рамках смежной ВКР. Она взаимодействует с базой данных на каждом шаге. Например, авторизация пользователя, загрузка контекста для GigaChat и сохранение сообщений. Работоспособность всего приложения напрямую зависит от надёжности, производительности и целостности данных.

В работе была реализована физическая схема базы данных: созданы таблицы, заданы первичные и внешние ключи, ограничения целостности и индексы. Проведено тестирование с оптимизацией узких мест на основе планировщика запросов. Разработанная база данных используется в самой платформе ViewTrain. Также полученные в работе решения подойдут разработчикам и другим веб-приложений, где нужно сохранять и обрабатывать данные о действиях пользователей.

Структурно работа состоит из трёх глав на 50 страницах и одного приложения на трёх страницах.

СОДЕРЖАНИЕ

АННОТАЦИЯ.....	1
ПЕРЕЧЕНЬ ИСПОЛЬЗУЕМЫХ СОКРАЩЕНИЙ	5
ВВЕДЕНИЕ.....	6
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ОПИСАНИЕ СТАРТАП-ПРОЕКТА VIEWTRAIN	8
1.1 ОПИСАНИЕ ПРОДУКТА VIEWTRAIN	8
1.2 СОСТОЯНИЕ РЫНКА	9
1.3 АНАЛИЗ КОНКУРЕНТОВ И ПОЗИЦИОНИРОВАНИЕ	9
1.4 БИЗНЕС-МОДЕЛЬ И ПОТЕНЦИАЛ МАСШТАБИРОВАНИЯ VIEWTRAIN	12
1.5 ОПИСАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ И УТОЧНЕНИЕ ТЕРМИНОЛОГИИ.....	15
1.6 МЕТОДОЛОГИЧЕСКИЕ ПОДХОДЫ К ПРОЕКТИРОВАНИЮ БАЗ ДАННЫХ	18
1.7 АНАЛИЗ СИСТЕМЫ VIEWTRAIN КАК ИСТОЧНИКА ТРЕБОВАНИЙ К ХРАНИЛИЩУ ДАННЫХ	20
1.8 ОБОСНОВАНИЕ ВЫБОРА PostgreSQL	21
1.9 АНАЛИЗ ТРЕБОВАНИЙ К БАЗЕ ДАННЫХ VIEWTRAIN	22
1.10 ПОСТАНОВКА ЗАДАЧИ НА ПРОЕКТИРОВАНИЕ.....	23
2 ПРОЕКТИРОВАНИЕ, РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ БАЗЫ ДАННЫХ VIEWTRAIN	25
2.1 КОНЦЕПТУАЛЬНАЯ МОДЕЛЬ ДАННЫХ.....	25
2.2 ЛОГИЧЕСКАЯ СХЕМА И НОРМАЛИЗАЦИЯ	26
2.3 ФИЗИЧЕСКАЯ СХЕМА В PostgreSQL: ТАБЛИЦЫ, ТИПЫ ДАННЫХ, ОГРАНИЧЕНИЯ	28
2.4 ДИАГРАММЫ СИСТЕМЫ	35
2.4.1 ER-ДИАГРАММА (КОНЦЕПТУАЛЬНЫЙ И ФИЗИЧЕСКИЙ УРОВНИ).....	35
2.4.2 ДИАГРАММА ВЗАИМОДЕЙСТВИЯ БД С СЕРВЕРНОЙ ЧАСТЬЮ.....	35
2.4.3 ДИАГРАММА ПОСЛЕДОВАТЕЛЬНОСТИ: СТАРТ СЕССИИ И ПЕРВЫЙ ХОД	37
2.5 СТРАТЕГИЯ ИНДЕКСИРОВАНИЯ	38
2.6 ТРАНЗАКЦИИ И ОБЕСПЕЧЕНИЕ ЦЕЛОСТНОСТИ ДАННЫХ.....	39
2.7 ПРОЦЕСС РЕАЛИЗАЦИИ И ПРИНЯТЫЕ РЕШЕНИЯ.....	40
2.8 СТРАТЕГИЯ ТЕСТИРОВАНИЯ	43
2.9 ФУНКЦИОНАЛЬНОЕ ТЕСТИРОВАНИЕ.....	43
2.10 НАГРУЗОЧНОЕ ТЕСТИРОВАНИЕ И ОПТИМИЗАЦИЯ.....	46
3 ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ ПРОЕКТА.....	51
3.1 ЗАТРАТЫ НА РАЗРАБОТКУ	51
3.2 ЗАТРАТЫ НА ЭКСПЛУАТАЦИЮ	51
3.3 ОЦЕНКА ЭФФЕКТИВНОСТИ ПРОЕКТНЫХ РЕШЕНИЙ	52
3.4 ОКУПАЕМОСТЬ.....	52
ЗАКЛЮЧЕНИЕ	54
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	56
ПРИЛОЖЕНИЯ.....	58

ПЕРЕЧЕНЬ ИСПОЛЬЗУЕМЫХ СОКРАЩЕНИЙ

AI – Artificial Intelligence – искусственный интеллект;

API – Application Programming Interface – программный интерфейс приложения;

БД – база данных;

ВКР – выпускная квалификационная работа;

DDL – Data Definition Language – язык определения данных;

ER – Entity-Relationship – сущность связь;

JWT – JSON Web Token – ключ аутентификации пользователя;

SHA256 – Secure Hash Algorithm Version 2 – безопасный алгоритм хеширования, версия 2;

ЗНФ – третья нормальная форма;

ACID – Atomicity Consistency Isolation Durability – набор требований к транзакционной системе;

MVCC – Multiversion Concurrency Control – многоверсионное управление конкурентным доступом;

СУБД – система управления базами данных;

SQL – Structured Query Language – язык запросов к базе данных;

UUID – Universally Unique Identifier – универсальный уникальный идентификатор;

WebSocket – протокол двусторонней связи через одно TCP-соединение.

ВВЕДЕНИЕ

С каждым годом рост требований к уровню подготовки IT-специалистов для прохождения успешного собеседования возрастает. Всё большую актуальность приобретают цифровые инструменты, позволяющие отрабатывать навыки прохождения технических собеседований в интерактивном формате. В связи с этим увеличивается интерес к программным решениям, способным имитировать реальный процесс собеседования и обеспечивать пользователю регулярную практику. Одним из таких решений является ViewTrain.

Это веб-приложение, которое помогает подготовиться к техническим собеседованиям. Диалог проходит с AI-интервьюером на базе GigaChat.

Проблема целевой аудитории состоит в том, что занятия с наставниками стоят дорого и не всегда регулярны. Не все платформы дают опыт живого диалога, а в сервисах парных тренировочных интервью необходим второй участник. ViewTrain позволяет закрыть этот пробел, он даёт неограниченную практику диалоговых интервью с ИИ-интервьюером и автоматической обратной связью. Похожих сервисов на рынке нет.

Серверная часть приложения разработана в рамках смежной ВКР группы. Она реализована с помощью FastAPI, JWT-авторизации, WebSocket-стриминга и интеграции GigaChat. Согласованность базы данных с проектом обеспечивается за счёт единого API-контракта, так как схема БД спроектирована под конкретные эндпоинты и паттерны запросов серверной части.

Актуальность выбора темы связана с производительностью и целостностью данных. Они, в свою очередь, напрямую определяют работоспособность ViewTrain как коммерческого продукта. Каждый запрос к модели формируется на сервере из истории сообщений, загружаемой из БД, так как GigaChat не хранит историю диалога. Медленный запрос в таблицу сообщений вызывает задержку перед каждым ответом. Это, в свою очередь,

напрямую ухудшает пользовательский опыт и критично для удержания аудитории.

Объектом исследования в данной работе выступает база данных ViewTrain на СУБД PostgreSQL.

Предметом исследования выступают методы проектирования реляционных схем, способы индексирования и механизмы обеспечения целостности данных при конкурентном доступе.

Цель работы — спроектировать и реализовать схему БД ViewTrain, которая закроет все сценарии работы серверной части, не создаст узких мест при нагрузке и будет оставаться поддерживаемой при росте системы.

Для достижения цели поставлены следующие задачи:

- провести анализ предметной области: определить сущности, связи и паттерны доступа к данным;
- объяснить выбор в пользу СУБД PostgreSQL;
- разработать концептуальную (ER-модель), логическую (с нормализаций до 3НФ) и физическую (DDL PostgreSQL) модели данных;
- разработать стратегию индексирования под конкретные паттерны запросов backend-сервиса;
- провести тестирование и оптимизацию узких мест по результатам планировщика запросов.

Практическая значимость работы заключается в спроектированной схеме, которая используется backend-сервисом ViewTrain в составе действующего стартап-проекта.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ОПИСАНИЕ СТАРТАП-ПРОЕКТА VIEWTRAIN

1.1 Описание продукта ViewTrain

ViewTrain — это веб-приложение для подготовки к техническим собеседованиям, в котором пользователь проходит диалоговую симуляцию интервью с AI-интервьюером на базе языковой модели GigaChat. Перед началом сессии пользователь выбирает тип интервью, например, алгоритмы, системное проектирование или поведенческое интервью. Далее пользователь выбирает уровень сложности (junior, middle или senior). После чего запускается симуляция живого диалога. ИИ задаёт вопрос, пользователь отвечает, модель уточняет и углубляется в тему так, как это сделал бы реальный интервьюер. По итогам сессии пользователь получает структурированную обратную связь по каждому ответу, а в личном кабинете отслеживает динамику прогресса от сессии к сессии.

Цель продукта — дать IT-специалистам возможность регулярно и неограниченно практиковать живой формат технического собеседования вне зависимости от расписания ментора, наличия второго участника или стоимости индивидуальных консультаций. В задачи продукта входят моделирование реалистичного интервью, оценка ответов пользователя с целью дать последующие рекомендации, а также сохранение истории прогресса пользователя. Уникальность ViewTrain складывается из трёх элементов. К ним относятся диалоговый формат интервью, ориентация на русскоязычную аудиторию с использованием российской языковой модели GigaChat. А также неограниченная доступность практики по подписке. На рынке отсутствуют прямые аналоги, сочетающие все три элемента одновременно. Это и формирует рыночное позиционирования продукта.

1.2 Состояние рынка

Прежде чем переходить к проектным решениям, важно зафиксировать состояние рынка, на который ориентирован продукт. В 2025 году глобальный рынок EdTech оценили примерно в 185 миллиардов долларов, а наиболее динамичным направлением внутри него стали решения на базе искусственного интеллекта: мировой объём ИИ в образовании вырос примерно с 6 до 8 миллиардов долларов и, по прогнозам, может превысить 32 миллиарда долларов к 2030 году.

Российский рынок также сохраняет рост. По данным Smart Ranking, по итогам 2025 года выручка топ-100 крупнейших edtech-компаний России составила 154 миллиарда рублей, увеличившись за год на 12%. При этом самую высокую динамику показывают сегменты, связанные с искусственным интеллектом, машинным обучением и анализом данных. Это совпадает с двумя ключевыми характеристиками ViewTrain — ориентацией на IT-специалистов и использованием языковой модели как ядра сервиса, что помещает продукт на пересечение двух наиболее активно растущих ниш российского EdTech.

Дополнительным фактором в пользу выбранной концепции служит отмечаемое аналитиками увеличение цикла принятия решения о покупке и рост числа запросов на рассрочку. Это подтверждает обоснованность freemium-модели, снижающей порог входа для пользователя. Объём рынка обеспечивает достаточный потенциал для роста стартапа на ранней стадии, а его ключевые тренды соответствуют проектным решениям, заложенным в основу ViewTrain.

1.3 Анализ конкурентов и позиционирование

Помимо общего описания продукта, необходимо рассмотреть существующие на рынке альтернативы и определить место ViewTrain среди них. Анализ конкурентов проводится в двух плоскостях: с точки зрения

рыночного позиционирования (формат, доступность, стоимость) и с точки зрения архитектурных особенностей хранения данных, поскольку именно архитектурная специфика определяет дальнейшие проектные решения по базе данных.

Обзор существующих платформ

На рынке инструментов для подготовки к техническим собеседованиям можно выделить несколько классов систем, различающихся как по формату взаимодействия с пользователем, так и по подходу к хранению данных.

LeetCode — наиболее распространённая платформа для практики алгоритмических задач. Система ориентирована на хранение большого каталога задач, решений пользователей и результатов автоматической проверки. Ключевая нагрузка ложится на таблицы решений и статистики, которые растут пропорционально числу пользователей и попыток. Диалогового сценария в системе нет — пользователь взаимодействует с задачей, а не с интервьюером. В этом ключевое отличие LeetCode от ViewTrain: необходимости хранить историю диалога и передавать её языковой модели при каждом запросе в LeetCode попросту не возникает.

Pramp — платформа для парных тренировочных интервью между пользователями в реальном времени. Система хранит данные о сессиях, участниках, вопросах и обратной связи. Поскольку интервью проводится между двумя людьми, а не с AI-интервьюером, история переписки не накапливается в базе в том объёме, который характерен для ViewTrain. Тем не менее сам сценарий хранения сессий и их статусов концептуально близок к рассматриваемой задаче.

AI-интервьюеры на базе больших языковых моделей — относительно новый класс систем, к которому относится ViewTrain. Их принципиальная особенность состоит в том, что языковая модель не хранит контекст диалога между запросами: каждый новый запрос к модели должен содержать всю предшествующую историю переписки. В итоге появляется специфический паттерн доступа к данным, из-за которого системе приходится выполнять

запрос всей истории сообщений текущей сессии перед каждым обращением к модели. При росте числа параллельных пользователей именно этот запрос становится узким местом системы. Готовых архитектурных шаблонов под этот паттерн нагрузки не существует, что делает невозможным простое заимствование решений из перечисленных выше систем.

Сравнение по рыночным критериям

Помимо архитектурных различий, конкурентов необходимо сопоставить по критериям, значимым для конечного пользователя: формату интервью, языку интерфейса, доступности, стоимости и регулярности практики. В Таблице 1.1 приведено сравнение ViewTrain с основными конкурентами по рыночным критериям.

Таблица 1.1 — Сравнение ViewTrain с основными конкурентами

Критерий	LeetCode	Pramp	ViewTrain
Формат	Алгоритмические задачи	Парные интервью с человеком	Диалог с AI-интервьюером
Язык интерфейса	Английский	Английский	Русский
Доступность по времени	24/7	По расписанию партнёра	24/7
Стоимость	Бесплатно / от \$35 в месяц	Бесплатно	Freemium / 499–799 Р в месяц
Регулярность практики	Высокая	Низкая	Высокая

Как видно из сравнения, ни один из конкурентов не сочетает одновременно диалоговый формат интервью, круглосуточную доступность и ориентацию на русскоязычную аудиторию. LeetCode развивает лишь навык решения алгоритмических задач, но не отрабатывает сам формат устного диалога с интервьюером. Pramp обеспечивает живой диалог, однако зависит от наличия второго участника и работает исключительно на английском языке.

Позиционирование ViewTrain

ViewTrain занимает свободную нишу на пересечении трёх характеристик: диалоговый формат интервью, круглосуточная доступность и ориентация на

русскоязычную аудиторию с использованием отечественной языковой модели GigaChat. Ни один из рассмотренных конкурентов не сочетает все три характеристики одновременно, что формирует позиционирование ViewTrain как доступного инструмента регулярной диалоговой практики на русском языке и определяет его потенциал коммерциализации в условиях российского рынка.

1.4 Бизнес-модель и потенциал масштабирования ViewTrain

Каждая сессия в ViewTrain представляет собой обращение к языковой модели, а значит — реальные расходы на вычислительные ресурсы. Эта черта проекта означает, что монетизация была продумана и заложена в основу продукта с самого начала, а не оставлена на потом. Полностью бесплатная модель в подобных условиях неустойчива, и ответ на вопрос «как именно сервис будет окупать себя» приходится закладывать ещё на стадии проектирования.

Базовым решением для ViewTrain выбрана модель freemium. Её суть состоит в том, что начальный уровень функциональности доступен любому пользователю без оплаты, а более глубокие сценарии работы с продуктом открываются по подписке. На бесплатном тарифе предусмотрено ограниченное число сессий в месяц — например, три — с доступом к основным типам интервью и базовым форматом разбора ответов. Этого объёма достаточно, чтобы пользователь сформировал собственное представление о продукте и принял осознанное решение о его дальнейшем использовании. Платная подписка снимает ограничение по количеству сессий. Открывает развёрнутый разбор с детальными рекомендациями к каждому ответу, доступ ко всем уровням сложности и индивидуальную аналитику прогресса.

Решение в пользу именно freemium связано с особенностями целевой аудитории. Значительную её долю составляют студенты и начинающие IT-специалисты — категория пользователей, для которой обязательная оплата ещё до знакомства с продуктом стала бы серьёзным препятствием. Бесплатные сессии устраняют этот барьер и позволяют человеку убедиться в полезности сервиса собственным опытом. В то же время те пользователи, которые относятся к подготовке к собеседованиям системно, довольно скоро оказываются ограничены лимитом бесплатного плана — и переход на подписку для них становится логичным следующим шагом.

Ценовая политика проекта выстраивается с ориентацией на российский рынок. Помесячная подписка предполагается в диапазоне 499–799 рублей, годовая — со значимой скидкой относительно простой суммы месячных платежей. Для контекста стоит сравнить эти суммы с альтернативными способами подготовки. Разовая консультация с карьерным консультантом обходится в 3 000–5 000 рублей, а полноценный курс по подготовке к техническим интервью — от 15 000 рублей и выше. На этом фоне ViewTrain обходится пользователю выгоднее. Причём подписка не накладывает ограничений ни по времени практики, ни по числу повторений.

Привлечение первых пользователей предполагается через несколько каналов, каждый из которых дополняет остальные. Наиболее естественный из них — контент-маркетинг. То есть размещение полезных материалов о подготовке к собеседованиям в профильных площадках вроде Habr, vc.ru и тематических Telegram-каналов для разработчиков, со встроенным упоминанием платформы. Второе направление — сотрудничество с техническими вузами: ViewTrain способен помочь студентам выпускных курсов на этапе подготовки к первому трудоустройству, а сами вузы заинтересованы в инструментах, которые улучшают показатели трудоустройства их выпускников. Третий канал — пользовательское сообщество. При условии, что платформа реально помогает успешно

проходить собеседования, начинает работать сарафанное радио, остающееся одним из самых эффективных каналов привлечения для любого продукта.

Перспективы масштабирования ViewTrain раскрываются сразу в нескольких плоскостях. Самая очевидная из них — расширение тематического охвата интервью. В текущей версии платформа поддерживает алгоритмические задачи, системное проектирование и поведенческие интервью. Этот список естественным образом расширяется в сторону смежных профессиональных областей. Подготовка для специалистов в области data science и машинного обучения, для продуктовых менеджеров, для QA-инженеров. Каждое новое направление приносит с собой новую аудиторию, не требуя при этом полной перестройки продукта.

Следующая плоскость развития — мобильное приложение. На него прекрасно ложится просмотр своих итогов, изучение советов. Появление мобильного клиента увеличило бы число точек контакта пользователя с платформой и повысило бы частоту возвращения к ней.

Отдельным направлением развития является выход в B2B-сегмент. Работодатели, нанимающие разработчиков, объективно заинтересованы в том, чтобы их кандидаты приходили на интервью подготовленными. Под этот запрос ViewTrain может предложить корпоративные лицензии: их могут использовать HR-подразделения — как для подготовки кандидатов перед интервью, так и для внутренней оценки уже работающих сотрудников. Принципиальное отличие этого канала монетизации от розничной подписки — заметно более высокий средний чек одного контракта.

К B2B-направлению логически примыкает идея интеграции ViewTrain с HR-сервисами и площадками поиска работы. Прямой переход к подготовке к собеседованию позволил бы платформе стать естественным элементом процесса трудоустройства.

Наконец, ещё одно направление масштабирования — локализация и выход на международный рынок. Здесь в пользу проекта работает сама природа технических интервью. Их формат универсален во всём мире, а

алгоритмические задачи почти одинаковые в любой стране. Адаптация интерфейса и его перевод на английский язык позволят выйти на мировой рынок. При этом значительная часть инфраструктуры продукта остаётся неизменной — фактически адаптации требуют только используемая языковая модель AI-интервьюера и тексты интерфейса.

Подводя итог, бизнес-модель ViewTrain опирается на freemium-подход с плавным и естественным переходом бесплатных пользователей в платных. Потенциал развития стартапа складывается из расширения тематик, выхода в мобильный формат, работы с корпоративным рынком, партнерства с HR-платформами, и в конечном итоге, выходе на глобальный уровень. Принципиально важно, что ни одно из этих направлений не требует перестройки существующего продукта и может реализовываться поэтапно, по мере роста аудитории и накопления ресурсов проекта.

1.5 Описание предметной области и уточнение терминологии

Данная работа находится на пересечении двух предметных областей. Это веб-приложения с диалоговым интерфейсом и реляционные системы хранения данных. До перехода к проектированию необходимо зафиксировать ключевые термины, которые используются на протяжении всей работы. Это нужно, чтобы избежать двусмысленности при описании архитектурных решений и их обоснований.

Реляционная база данных и СУБД

База данных — это место, где данные хранятся в строгом порядке. Чтобы с ними можно было работать: добавлять, искать и менять, существует СУБД. Система управления базами данных следит за тем, чтобы данные не разъехались, когда к базе одновременно обращаются несколько пользователей. Самый популярный способ организовать данные внутри — реляционный. Эдгар Кодд предложил его в 1970 году. Данные раскладываются по таблицам, а таблицы связываются друг с другом через

ключи. В данной работе в качестве СУБД используется PostgreSQL — объектно-реляционная система с открытым исходным кодом, поддерживающая расширенные типы данных, пользовательские перечисления, массивы и развитый механизм индексирования.

ACID и транзакции

Транзакция — логически неделимая последовательность операций с базой данных, которая либо выполняется целиком, либо не выполняется вовсе. Надёжность транзакций описывается набором свойств ACID.

Атомарность означает, что все операции транзакции рассматриваются как единое целое. К примеру, если одна из них завершилась ошибкой, то все изменения откатятся. Согласованность же даёт гарантии, что после завершения транзакции база данных переходит из одного корректного состояния в другое, не нарушая ни одного ограничения целостности. Независимость параллельных транзакций обеспечивается за счёт изолированности, промежуточные состояния одной транзакции не видны другим. Если транзакция успешно завершилась — её результат никуда не денется, даже если система упадёт. Это и есть долговечность.

Для ViewTrain свойство атомарности критично в трёх сценариях: при завершении сессии собеседования, старта сессии и обновлении токена авторизации. Все они требуют выполнения нескольких операций как единого целого.

MVCC

Многоверсионное управление конкурентным доступом (MVCC) — механизм, при котором каждая транзакция работает со своим согласованным снимком данных на момент её начала. Так вместо блокировки строк при чтении PostgreSQL создаёт новую версию строки при каждом изменении, сохраняя старую для активных читателей. Это означает, что читатели не блокируют писателей и наоборот.

Для ViewTrain MVCC особенно важен — в системе постоянно что-то происходит параллельно. Пока один пользователь загружает историю

переписки, чтобы передать её в GigaChat, другой в это же время отправляет новое сообщение в ту же таблицу. Если бы не MVCC, то они бы мешали друг другу: возникали бы очереди ожидания, и приложение начинало бы заметно тормозить.

Нормализация и нормальные формы

Нормализация — это процесс приведения таблиц в порядок. Главная цель — убрать лишние данные и избежать ситуаций, когда при добавлении, изменении или удалении записи что-то ломается или расходится. Степень нормализации описывается нормальными формами. Их несколько, каждая строже предыдущей. Первая нормальная форма заключается в том, что в каждой ячейке должно быть одно значение, никаких списков и повторяющихся групп. Вторая добавляет: каждое поле таблицы должно зависеть от первичного ключа целиком, а не от его части. Третья нормальная форма выполняется, если схема удовлетворяет второй и ни один неключевой атрибут не зависит транзитивно от первичного ключа через другой неключевой атрибут.

Query-driven design

Query-driven design — методологический подход к проектированию схемы базы данных, при котором структура таблиц и индексов определяется не абстрактной моделью данных, а конкретными запросами, которые приложение будет выполнять в процессе работы. В отличие от классического подхода, где сначала строится полная модель данных, а про оптимизацию вспоминают уже потом — здесь всё наоборот: сначала думаем о запросах, и только потом проектируем под них структуру.

Как отмечает Клеппман, в высоконагруженных системах паттерны доступа к данным должны быть известны до начала проектирования схемы, поскольку структурные изменения в работающей системе обходятся значительно дороже, чем правильное решение, принятое с первого раза.

Индексирование

Индекс — это вспомогательная структура данных. Она ускоряет поиск и сортировку записей с помощью хранения упорядоченных ссылок на строки таблицы. PostgreSQL поддерживает несколько типов индексов. В данной работе используется B-Tree — сбалансированное дерево поиска. Оно оптимально для запросов с условиями равенства и диапазона. Индексы создаются автоматически для первичных ключей и ограничений уникальности; все остальные индексы в схеме ViewTrain созданы явно на основе анализа паттернов доступа.

1.6 Методологические подходы к проектированию баз данных

Прежде чем приступать к проектированию собственного решения, необходимо рассмотреть, какие методологические подходы к построению схем реляционных баз данных рекомендует научное сообщество. Это позволит сформировать теоретическую основу для проектных решений, принятых в работе.

Подходы к проектированию реляционных баз данных

В литературе есть два основных подхода к проектированию схемы базы данных и, как правило, их рассматривают как противоположные.

В работах Элмасри и Наватэ описан классический подход. Он предполагает последовательное движение от концептуальной модели к физической. Для начала создается полная структура базы данных, которая покажет все сущности и связи между ними. Затем она приводится до 3НФ и только после этого, создается физическая схема. Плюсы этого подхода — строгость и воспроизводимость, а недостаток — риск реализовать нормализованную схему, которая может плохо работать под реальной нагрузкой. Обратная сторона классического подхода — это Query-driven подход, описанный в подразделе 1.5. Его достоинства — это возможность прогнозировать время отклика. Также он подходит для непропорциональной

нагрузки на базу данных, например, в ситуациях, где операций чтения больше, чем записей.

В данном проекте сочетаются оба подхода. Сначала анализируются основные возможные запросы от приложения, а затем выстраивается концептуальная и логическая модель базы данных. При этом любое отклонение от 3НФ допускается только в случае, если база будет работать быстрее.

Подходы к индексированию

Винанд обозначил основной принцип индексирования. Он заключается в том, что эффективность индекса определяется высокой селективностью данных. То есть, в ситуациях, когда он позволяет исключить большую часть строк таблицы до их фактического чтения. Если создать индекс для поля с низкой селективностью, это не даст ощутимого прироста к скорости. Также это может снизить скорость добавления и исправления данных в таблице, так как при каждой операции индекс будет перестраиваться.

Карвин указывает на типичный антипаттерн. Создание индекса на каждом столбце таблицы в надежде ускорить любой возможный запрос. На практике это приводит к обратному эффекту — деградации производительности записи при отсутствии значимого ускорения чтения. Поскольку большинство созданных индексов планировщик запросов игнорирует.

В данной работе стратегия индексирования формируется строго на основе выявленных паттернов доступа: создаётся минимально необходимый набор индексов, каждый из которых закрывает конкретный запрос из сценариев использования системы.

1.7 Анализ системы ViewTrain как источника требований к хранилищу данных

В подразделе 1.6 был обоснован выбор комбинированного подхода к проектированию. Чтобы применить этот подход на практике, необходимо детально описать саму систему и выявить все сценарии, в которых серверная часть обращается к базе данных. Именно эти сценарии станут основой для всех последующих проектных решений — выбора СУБД, структуры таблиц и стратегии индексирования. Поскольку ViewTrain является стартап-проектом на ранней стадии, корректное определение этих сценариев на этапе проектирования критично: ошибки в архитектуре хранилища, обнаруженные уже после запуска, обходятся стартапу значительно дороже, чем правильное решение, принятое заранее.

Проектирование базы данных стоит начинать с определения запросов к ней, а не с ее сущностей. Такой подход называется *query-driven design*. При создании схемы учитывается то, как приложение работает с данными на практике. За основу берутся основные сценарии из смежной ВКР. Обращение серверной части приложения к базе происходит в следующих сценариях.

Первый сценарий, при авторизации — поиск пользователя по почте, поиск токена авторизации, создание новой записи токена и отзыв старой.

Второй сценарий, при каждом ходе в сессии — загрузка всей истории сообщений сессии в хронологическом порядке, сохранение нового сообщения от пользователя и AI.

Третий сценарий, при загрузке страниц — отфильтрованная выборка сессий пользователя по статусу и типу, а также выборка переписки конкретной сессии.

Проанализировав все сценарии, следует ключевой вывод: самый частый запрос к базе данных — запрос истории сообщений сессии. Он выполняется перед каждым обращением к GigaChat, то есть при каждом новом сообщении пользователя. Если взять 100 подобных параллельных запросов, то при

деградации приложение станет очень медленным до достижения действительно больших объемов. Для стартапа на стадии активного роста аудитории такая деградация означает прямую угрозу удержанию пользователей, поэтому именно этот сценарий лёг в основу всех ключевых решений по индексированию.

1.8 Обоснование выбора PostgreSQL

Выбирая СУБД рассматривались три варианта. Первый — PostgreSQL (реляционная), второй — MongoDB (нереляционная) и третий — Redis (ключ-значение). Критерии для сравнения формировались исходя из требований стартап-проекта: надёжность хранения пользовательских данных, способность выдерживать рост нагрузки при масштабировании сервиса и минимизация технических рисков на ранней стадии развития продукта. В Таблице 1.2 приведено сравнение по критериям, значимым для ViewTrain.

Таблица 1.2 — Сравнение СУБД по критериям, значимым для ViewTrain

Критерий	PostgreSQL	MongoDB	Redis
ACID-транзакции	Полные	Только с версии 4.0	Атомарные команды
JOIN между таблицами	Нативный	Денормализовывать или собирать вручную	Не поддерживается
MVCC (параллельность)	Встроенный	Блокировка на уровне документа	Однопоточный
Схема данных	Строгая	Гибкая	Ключ-значение
Агрегации	Да	Aggregation Pipeline	Ограниченно
Долговременное хранение	Да	Да	Опционально

PostgreSQL был выбран по совокупности особенностей. Во-первых, данные ViewTrain имеют стабильную структуру и четкие связи с таблицами: пользователь, сессия, сообщение. MongoDB имеет избыточную гибкость. Во-вторых, требуются полные ACID-транзакции для завершения сессии — атомарная операция обновления сессии и добавление последнего сообщения. В-третьих, MVCC PostgreSQL позволяет вести параллельную работу

множества сессий без взаимных блокировок: читатели не блокируют писателей в той же таблице. Redis используется в архитектуре только как модуль управления соединениями — это решение уровня серверной части, не уровня основной БД. Дополнительным аргументом в пользу PostgreSQL стало то, что это зрелая открытая СУБД без лицензионных отчислений, что для стартапа на ранней стадии означает отсутствие затрат на лицензии и независимость от конкретного коммерческого вендора.

1.9 Анализ требований к базе данных ViewTrain

В качестве основной СУБД был выбран PostgreSQL. Дальше нужно определить, что именно он должен обеспечивать в рамках конкретного проекта. На основе сценариев из подраздела 1.7 сформированы функциональные и нефункциональные требования к базе данных. Они служат критериями приёмки для всех проектных решений и тестирования раздела. В Таблице 1.3 представлены функциональные требования к БД ViewTrain.

Таблица 1.3 — Функциональные требования к БД ViewTrain

Идентификатор	Описание
ФТ-БД-01	Хранение пользователей: email (уникален), хэш пароля, имя пользователя, дата регистрации, флаг активности
ФТ-БД-02	Хранение refresh-токенов: SHA-256-хэш (уникален), вторичный ключ на пользователя с CASCADE DELETE, срок истечения, флаг отзыва
ФТ-БД-03	Хранение сессий: тип интервью, уровень сложности, статус (created / active / completed / abandoned), временные метки
ФТ-БД-04	Хранение сообщений: роль (user / assistant), текстовое содержимое, временная метка, вторичный ключ на сессию с CASCADE DELETE
ФТ-БД-05	Хранение каталога вопросов: тип, уровень сложности, текст, теги (массив строк)
ФТ-БД-06	Каскадное удаление: удаление пользователя удаляет все его сессии и сообщения

Нефункциональные требования задают измеримые критерии производительности и целостности, которым должна удовлетворять схема при реалистичной нагрузке. Каждый критерий сформулирован в виде конкретного

порогового значения. Это позволяет однозначно оценить результат в ходе тестирования. В Таблице 1.4 представлены нефункциональные требования к БД ViewTrain.

Таблица 1.4 — Нефункциональные требования к БД ViewTrain

Идентификатор	Критерий приемлемости
НФТ-БД-01	SELECT полной истории сессии (50 сообщений) выполняется не более чем за 50 мс при объёме таблицы messages 1 млн записей
НФТ-БД-02	INSERT нового сообщения выполняется не более чем за 10 мс
НФТ-БД-03	Выборка списка сессий пользователя с фильтрацией по статусу выполняется менее чем за 200 мс при объёме таблицы sessions 50 000 записей
НФТ-БД-04	Поиск refresh-токена по хэшу выполняется не более чем за 50 мс

Сформулированные требования будут использованы как критерии приёмки на этапе тестирования в разделе 2.

1.10 Постановка задачи на проектирование

Требования, сформулированные в подразделе 1.9, определяют критерии приёмки результата. Данный подраздел фиксирует методы достижения этих критериев и последовательность работ.

За основу взят комбинированный подход. Сначала разбираемся, как приложение будет обращаться к данным, и только потом строим модели. Однако нормализация до третьей нормальной формы остаётся обязательной. Отступить от неё можно, только если есть конкретное измеримое обоснование через производительность, и такое отступление обязательно фиксируется в документации.

Проектирование было разбито на четыре этапа. Сначала строится концептуальная модель — пять сущностей и связи между ними, без привязки к какой-либо конкретной СУБД. Затем проводится нормализация до третьей нормальной формы и фиксируются все отклонения от неё. На третьем этапе появляется физическая схема PostgreSQL: типы данных, таблицы,

ограничения и правила каскадного удаления. Последний же этап — разработка стратегии индексирования.

В область работы входит всё, что напрямую касается того, как устроена база данных ViewTrain: как данные описаны на концептуальном, логическом и физическом уровнях, как обеспечивается их целостность и как выстроено индексирование. Результат проектирования рассматривается как готовый инфраструктурный компонент стартап-проекта ViewTrain, пригодный для непосредственного использования backend-сервисом и дальнейшего масштабирования при росте пользовательской базы.

2 ПРОЕКТИРОВАНИЕ, РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ БАЗЫ ДАННЫХ VIEWTRAIN

2.1 Концептуальная модель данных

Перед тем чтобы проектировать таблицы и писать DDL-скрипты, необходимо разобраться, что вообще хранится в системе и как эти данные будут связаны между собой. Для этого строится концептуальная модель — она должна описывать предметную область в общих чертах, без привязки к какой-либо СУБД. На данном этапе очень важно понять общую логику, а не детали реализации. Концептуальная модель ViewTrain отражает основные сущности, с которыми работает стартап-продукт, — пользователей, их сессии собеседований, переписку с AI-интервьюером и каталог вопросов, — и формирует основу для всей последующей реализации. При анализе особенностей системы ViewTrain получилось выделить пять основных сущностей, представленных на Рисунке 2.1.

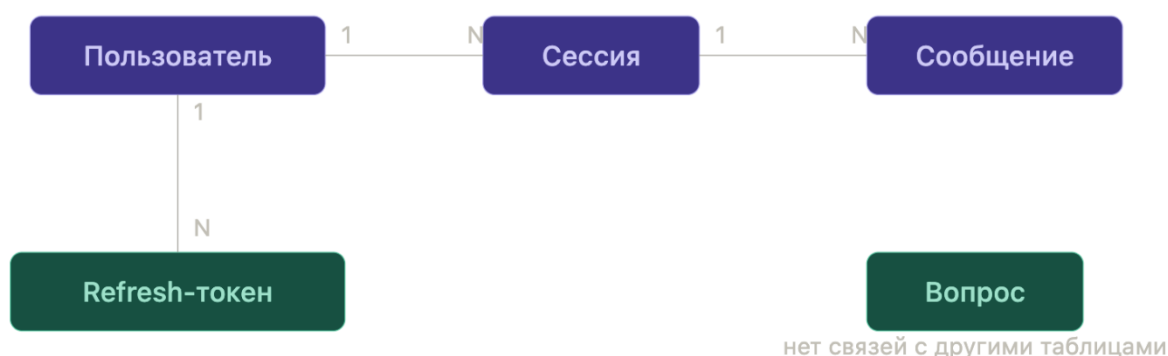


Рисунок 2.1 — Концептуальная ER-диаграмма базы данных ViewTrain

Пользователь — это человек, который зарегистрировался в системе и проходит собеседование, а также имеет персональные данные для входа в свой личный кабинет. С одним пользователем связаны все его сессии собеседований и токены авторизации, поэтому связь здесь — один ко многим.

Сессия — сессия собеседования, принадлежащая конкретному пользователю, и определяется типом интервью и уровнем сложности. У сессии есть статус, который меняется в зависимости от события: сначала сессия создаётся, затем становится активной, а в конце либо завершается, либо прерывается. За счёт меток старта и окончания можно посчитать, сколько времени пользователь провёл на собеседовании.

Сообщение — элемент переписки, который относится к роли человека или AI-интервьюера. Именно из всех сообщений текущей сессии собирается контекст, который при каждом новом запросе передаётся в GigaChat, ведь модель не хранит историю диалога самостоятельно.

Refresh-токен — служебная сущность, связанная с авторизацией и хранением токена. В базе хранится не сам токен, а его SHA-256-хэш сделано это было специально в целях безопасности. Если данные утекут, злоумышленник получит только хэш, а не рабочий токен. У каждого токена есть срок жизни, после которого он становится недействительным, и флаг отзыва — на случай если нужно принудительно завершить сессию пользователя раньше времени.

Вопрос — хранилище вопросов, из которого берётся задание при создании новой сессии. Эта сущность специально не связана с остальными. Когда сессия стартует, вопрос просто копируется как первое сообщение с ролью интервьюера. Благодаря этому можно свободно редактировать и обновлять каталог вопросов, не затрагивая историю уже завершённых сессий.

2.2 Логическая схема и нормализация

Когда концептуальная модель готова и понятно, какие сущности существуют в системе, можно переходить к следующему шагу — логической модели. На этом этапе каждая сущность превращается в таблицу, добавляются конкретные поля, определяются их типы и уточняется, как таблицы связаны друг с другом. Параллельно данные проверяются на избыточность и

дублирование — этот процесс называется нормализацией. Нормализация позволяет устранить аномалии при добавлении, изменении и удалении данных, а также снизить вероятность рассогласования информации в разных частях базы. Все пять таблиц ViewTrain приведены к третьей нормальной форме, однако в двух местах сделаны осознанные отступления от строгих правил — каждое из них задокументировано и обосновано соображениями производительности.

Первая нормальная форма требует, чтобы каждое поле таблицы хранило одно атомарное, неделимое значение, и не допускает повторяющихся групп. В целом это условие по всей схеме соблюдено, однако есть одно специальное исключение: поле `tags` в таблице каталога вопросов объявлено как массив строк. Формально это нарушает строгую первую нормальную форму, которая предписывает выносить подобные данные в отдельную таблицу. Тем не менее PostgreSQL умеет работать с массивами напрямую — фильтровать записи по тегам можно без разбора строк и без дополнительного соединения с отдельной таблицей. Для каталога такого объёма это просто удобнее — и в написании запросов, и в последующей поддержке.

Вторая нормальная форма соблюдена полностью и без исключений: у каждой из пяти таблиц одиночный первичный ключ, а значит, ситуация, при которой неключевой атрибут зависел бы лишь от части составного ключа, в принципе невозможна.

Третья нормальная форма потребовала небольшой, но важной правки на этапе проектирования. В исходной версии схемы в таблице сессий хранилось поле с именем пользователя — предполагалось использовать его для отображения в истории. Однако имя пользователя — это данные о пользователе, а не о сессии. Хранить его в таблице сессий не имеет смысла. Поле убрали, а имя теперь просто берётся из таблицы пользователей по мере необходимости.

Вместе с тем в схеме присутствует одно осознанное отступление от третьей нормальной формы. В таблице сессий оставлено поле `message_count`

— денормализованный счётчик сообщений. По строгим правилам нормализации это вычисляемое значение: его можно получить агрегационным запросом `COUNT(*)` по таблице сообщений. Однако выполнять такую агрегацию при каждой загрузке списка сессий означает проходить по потенциально большому числу строк при каждом обращении. Вместо этого было решено обновлять счётчик автоматически триггером, который срабатывает при каждой вставке новой записи в таблицу сообщений. В результате чтение списка сессий всегда выполняется за фиксированное время, независимо от того, сколько сообщений накопилось в базе.

2.3 Физическая схема в PostgreSQL: таблицы, типы данных, ограничения

После концептуальной и логической моделей приходит время физической схемы. Здесь уже создаются таблицы, а каждое поле получает конкретный тип и прописываются ограничения. То, что решается на этом этапе, потом либо работает быстро и надёжно, либо создаёт проблемы в самый неподходящий момент.

В Листинге 2.1 представлен DDL-скрипт для создания таблицы `users`.

Листинг 2.1 — DDL таблицы users

```
CREATE TABLE users (  
    id                UUID          PRIMARY KEY DEFAULT gen_random_uuid(),  
    email             VARCHAR(255)  UNIQUE NOT NULL,  
    username          VARCHAR(100)  NOT NULL,  
    hashed_password   VARCHAR(255)  NOT NULL,  
    created_at        TIMESTAMPTZ   NOT NULL DEFAULT NOW(),  
    is_active         BOOLEAN       NOT NULL DEFAULT TRUE  
);
```

Таблица `users` хранит всех зарегистрированных пользователей системы. В качестве первичного ключа выбран `UUID`, который генерируется функцией `gen_random_uuid()`. Это сознательный отказ от привычного автоинкрементного идентификатора, так как `UUID` можно сформировать прямо на стороне приложения ещё до того, как запись попадёт в базу — это убирает лишнее обращение к БД. Кроме того, порядковый числовой

идентификатор косвенно раскрывает, сколько пользователей зарегистрировано в системе, UUID такой информации не даёт.

Поле email, обязательное и уникальное, объявлено как varchar(255). Стоит учитывать, что PostgreSQL автоматически создаёт индекс на уникальное поле, так что дополнительно его создавать не нужно, а поиск пользователя по почте при авторизации будет быстрым из коробки.

Поле username — имя пользователя, обязательное, varchar(100).

Пароль хранится в поле hashed_password — тоже varchar(255) и обязательное. Сам пароль в базе никогда не появляется, только его хэш, который занимает 60 символов. Запас до 255 символов оставлен намеренно, так как если в будущем алгоритм хэширования сменится на другой с более длинным выводом, менять схему не придётся.

Дата создания, обязательное — в таблице хранятся с типом timestamptz — это вариант с поддержкой часового пояса. Такой подход исключает путаницу с временем для пользователей из разных регионов.

Флаг is_active, обязательное и объявлен как boolean со значением по умолчанию TRUE.

В Листинге 2.2 представлен DDL-скрипт для создания ENUM-типов и таблицы sessions.

Листинг 2.2 — DDL ENUM-типов и таблицы sessions

```
CREATE TYPE interview_type AS ENUM
    ('algorithms', 'system_design', 'behavioral');

CREATE TYPE difficulty_level AS ENUM ('junior', 'middle', 'senior');

CREATE TYPE session_status AS ENUM
    ('created', 'active', 'completed', 'abandoned');

CREATE TABLE sessions (
    id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id           UUID                NOT NULL
    REFERENCES users(id) ON DELETE CASCADE,
    interview_type    interview_type     NOT NULL,
    difficulty         difficulty_level   NOT NULL,
    status            session_status     NOT NULL DEFAULT 'created',
    message_count     INTEGER            NOT NULL DEFAULT 0,
    started_at        TIMESTAMPTZ,
    ended_at          TIMESTAMPTZ,
    created_at        TIMESTAMPTZ       NOT NULL DEFAULT NOW()
);
```

Таблица `sessions` хранит все сессии собеседований в системе. Первичный ключ — `UUID`, генерируемый через `gen_random_uuid()`, по той же причине что и в таблице `users`.

Поле `interview_type` – обязательное, хранит тип интервью и объявлено как пользовательский перечисляемый тип, который допускает следующие значения: `algorithms`, `system_design` и `behavioral`.

Поле `difficulty`, обязательное — работает по той же логике, перечисляемый тип с тремя уровнями сложности: `junior`, `middle` и `senior`.

Поле `status`, обязательное, показывает текущее состояние сессии, тоже перечисляемый тип с ограничением. Позволяет вносить только такие статусы: `created`, `active`, `completed` и `abandoned`, где значение по умолчанию `created`.

Поле `message_count` — `integer`, обязательное, по умолчанию 0. Это тот самый денормализованный счётчик, про который уже шла речь в разделе логической модели.

Поля `started_at` и `ended_at` фиксируют момент старта и завершения сессии. В отличие от `created_at`, они не заполняются автоматически и могут быть `NULL` — сессия уже существует в базе, но собеседование ещё не началось.

Поле `created_at` работает так же, как в таблице `users`: `timestampz`, обязательное с `DEFAULT NOW()`.

В Листинге 2.3 представлен DDL-скрипт для создания таблицы `messages`.

Листинг 2.3 — DDL таблицы `messages`

```
CREATE TYPE message_role AS ENUM ('user', 'assistant');

CREATE TABLE messages (
    id          BIGSERIAL      PRIMARY KEY,
    session_id  UUID           NOT NULL
                REFERENCES sessions(id) ON DELETE CASCADE,
    role        message_role  NOT NULL,
    content     TEXT           NOT NULL,
    created_at  TIMESTAMPTZ    NOT NULL DEFAULT NOW()
);
```

Таблица `messages` хранит всю переписку внутри сессий. В отличие от предыдущих таблиц, первичный ключ здесь `BIGSERIAL`, то есть обычный автоинкрементный числовой идентификатор.

Поле `session_id` — обязательное, служит ссылкой на сессию, тип `UUID` с каскадным удалением. Удаляется сессия — удаляется вся её переписка.

Поле `role`, обязательное, перечисляемый тип, допускает два значения `user`(реплика пользователя) и `assistant`(ответ AI-интервьюера).

Поле `content`, обязательное, было объявлено как `TEXT` без ограничения длины, так как ответы `GigaChat` могут быть развёрнутыми и был бы риск обрезки данных.

Поле `created_at`, обязательное, тип `timestampz` со значением по умолчанию равным текущей дате и времени.

В Листинге 2.4 представлен DDL-скрипт для создания таблицы `refresh_tokens`.

Листинг 2.4 — DDL таблицы `refresh_tokens`

```
CREATE TABLE refresh_tokens (  
    id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),  
    user_id     UUID          NOT NULL  
        REFERENCES users(id) ON DELETE CASCADE,  
    token_hash  VARCHAR(64)   UNIQUE NOT NULL,  
    expires_at  TIMESTAMPTZ   NOT NULL,  
    revoked     BOOLEAN       NOT NULL DEFAULT FALSE,  
    created_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW(),  
    CONSTRAINT chk_expires CHECK (expires_at > created_at)  
);
```

Таблица `refresh_tokens` хранит токены для обновления доступа к системе. Первичный ключ — `UUID`, генерируемый через `gen_random_uuid()`, по той же причине что и в таблице `users`.

Поле `user_id`, обязательное, служит ссылкой на пользователя — тип `UUID` с каскадным удалением. Удаляется пользователь — удаляются все его токены.

Поле `token_hash`, обязательное и уникальное с типом `varchar(64)`. В базе намеренно хранится не сам токен, а его SHA-256-хэш, который занимает ровно

64 символа. Если данные из базы утекут, то реальные токены никто не сможет увидеть.

Поле `expires_at`, обязательное с типом `timestampz`, фиксирует момент истечения токена.

Поле `revoked`, обязательное и объявлено как `boolean` значением по умолчанию `FALSE`. Оно позволяет явно инвалидировать токен при выходе пользователя из системы — без ожидания истечения срока.

Поле `created_at` работает так же, как в предыдущих таблицах. Тип `timestampz`, обязательное со значением текущей даты и времени по умолчанию.

В Листинге 2.5 представлен DDL-скрипт для создания таблицы `questions`.

Листинг 2.5 — DDL таблицы `questions`

```
CREATE TABLE questions (  
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),  
  interview_type    interview_type     NOT NULL,  
  difficulty        difficulty_level   NOT NULL,  
  content           TEXT                NOT NULL,  
  tags             TEXT[]  
);
```

Таблица `questions` хранит банк вопросов. Первичный ключ — `UUID`, генерируемый через `gen_random_uuid()`, по той же причине что и в остальных таблицах.

Поле `interview_type`, обязательное, перечисляемый тип и имеет значения `algorithms`, `system_design` и `behavioral`.

Поле `difficulty`, обязательное, тоже перечисляемый тип с тремя уровнями сложности: `junior`, `middle` и `senior`.

Поле `content`, обязательное, тип `TEXT` без ограничения длины, хранит текст вопроса.

Поле `tags` — массив строк `text[]`, единственное необязательное поле в таблице. Если теги указаны, по ним можно фильтровать вопросы напрямую

через оператор @>, без разбора строки. Если тегов нет — поле просто остаётся пустым.

Полный DDL-скрипт схемы базы данных ViewTrain приведён в приложении А.

Физическая ER-диаграмма базы данных ViewTrain, отражающая реализацию схемы с типами данных, первичными и внешними ключами, представлена на Рисунке 2.2.

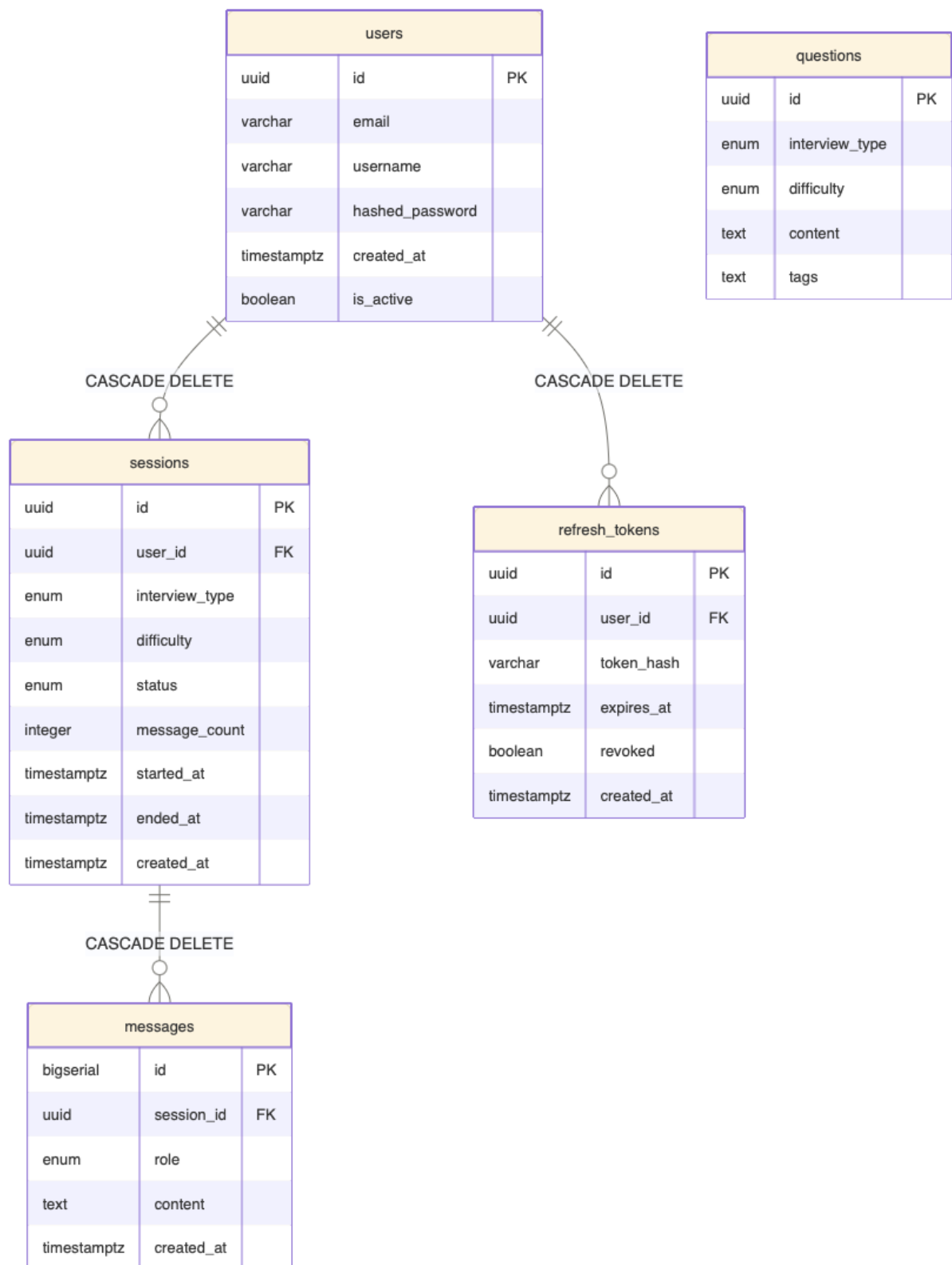


Рисунок 2.2 — Физическая ER-диаграмма базы данных ViewTrain

Как видно на диаграмме, физическая схема полностью соответствует концептуальной модели.

2.4 Диаграммы системы

2.4.1 ER-диаграмма (концептуальный и физический уровни)

На концептуальной ER-диаграмме (Рисунок 2.1) показано, какие сущности существуют в системе и как они связаны между собой — без привязки к конкретной СУБД и технических подробностей. Физическая ER-диаграмма (Рисунок 2.2) представляет уже конкретную реализацию схемы: с названиями таблиц, типами данных, первичными и внешними ключами, ограничениями и правилами каскадного удаления.

Четыре из пяти таблиц связаны между собой через внешние ключи. Пользователь — это центральная сущность, от которой тянутся все остальные: его сессии и его refresh-токены. Внутри каждой сессии хранятся сообщения — то, что было написано в ходе конкретного собеседования. Данная структура удобна тем, что при удалении пользователя не нужно вручную удалять все связанные с ним данные. Из-за каскадного удаления PostgreSQL сам пройдёт по цепочке и уберёт всё: сессии, сообщения и токены. С таблицей каталога вопросов всё немного по-другому. Она не связана с остальными таблицами через внешние ключи — и это сделано осознанно. Когда пользователь начинает новую сессию, из каталога просто берётся подходящий вопрос и копируется в таблицу сообщений как первое сообщение от интервьюера. После этого каталог никак не задействован. Такой подход позволяет спокойно менять и дополнять банк вопросов, не беспокоясь о том, что это как-то затронет историю старых сессий.

2.4.2 Диаграмма взаимодействия БД с серверной частью

На Рисунке 2.3 показано, как серверная часть обращается к базе данных в зависимости от того, что делает пользователь. Это удобно тем, что сразу видно, какие запросы к БД стоят за каждым действием в приложении.

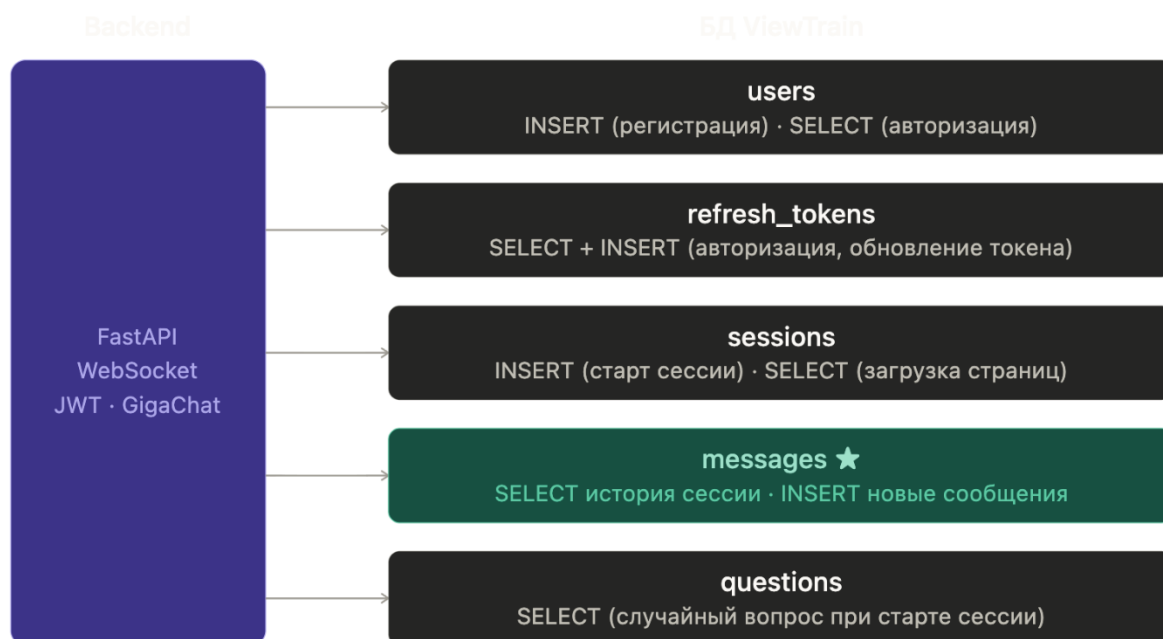


Рисунок 2.3 — Диаграмма взаимодействия backend-сервиса и БД ViewTrain

При регистрации создаётся новая запись в таблице пользователей. При авторизации сервер ищет пользователя по email, и здесь пригождается индекс, который PostgreSQL создаёт автоматически на уникальное поле. За счет этого поиск получается быстрым без каких-либо дополнительных настроек.

Самый частый сценарий — это каждый ход в сессии. Перед тем как отправить запрос в GigaChat, сервер загружает всю историю переписки из таблицы сообщений, потому что модель не помнит предыдущих реплик и каждый раз получает контекст заново. После того как GigaChat возвращает ответ, он сохраняется в таблицу сообщений вместе с репликой самого пользователя. Именно под этот сценарий и создавался составной индекс `idx_messages_session_id` для того, чтобы выборка истории работала быстро даже при большом объёме данных.

При загрузке страницы пользователя выполняется выборка списка сессий с фильтрацией по `user_id` и статусу.

2.4.3 Диаграмма последовательности: старт сессии и первый ход

На Рисунке 2.4 показана последовательность событий при старте сессии и первом сообщении пользователя.

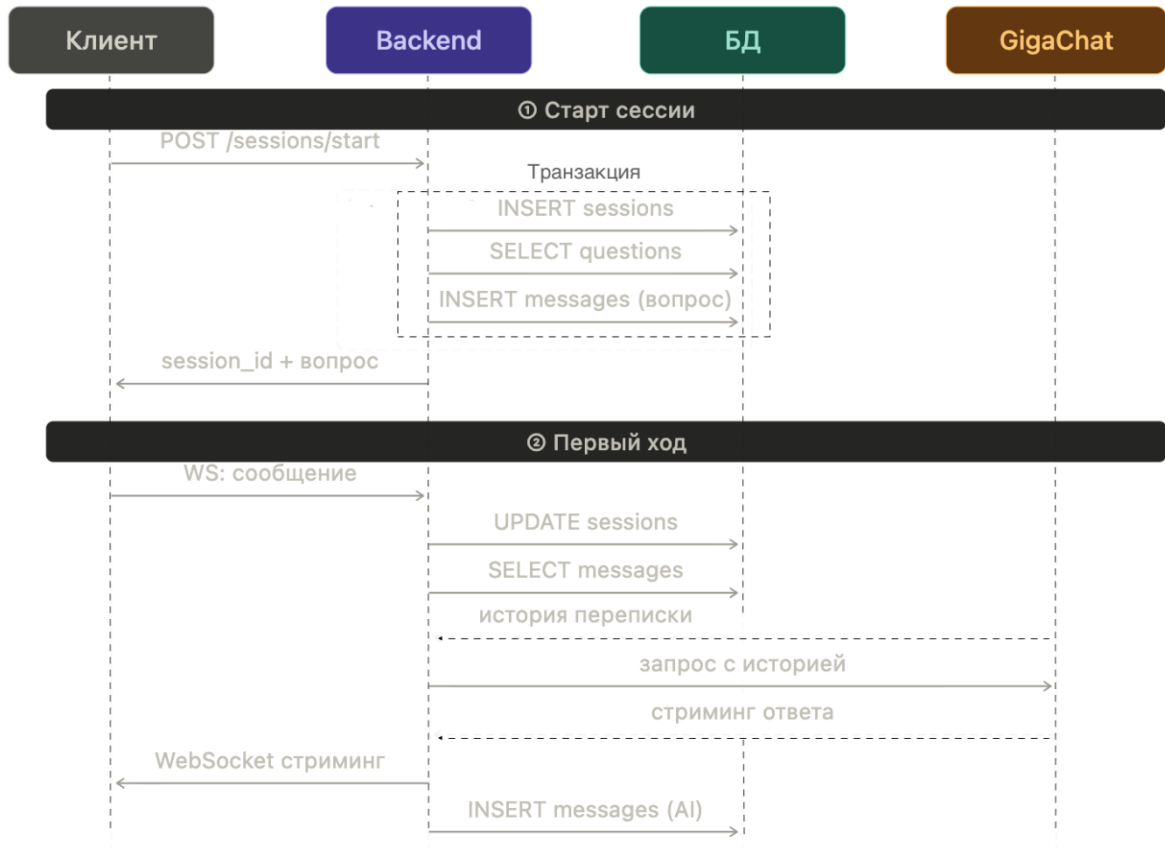


Рисунок 2.4 — Диаграмма последовательности: старт сессии и первый ход

При старте сессии пользователя запускается сразу три действия в рамках одной транзакции. В базе создаётся запись о новой сессии со статусом «создана», из каталога случайным образом выбирается вопрос подходящего типа и уровня сложности, после чего этот вопрос записывается в таблицу сообщений как первая реплика интервьюера.

После того как пользователь отправляет своё первое сообщение, сессия переходит в статус «активна» и фиксируется время её начала. Затем из базы загружается вся история переписки и передаётся в GigaChat в качестве контекста. Пока модель генерирует ответ, данные стримятся на клиент. После того как стриминг завершается, ответ сохраняется в таблицу сообщений, а счётчик сообщений в записи сессии увеличивается на единицу.

2.5 Стратегия индексирования

PostgreSQL автоматически создаёт B-Tree индексы для первичных ключей и ограничений на уникальность, остальные же необходимые индексы приходится создавать самостоятельно и главное осознанно, так как излишние индексы будут замедлять манипуляции с данными. Именно поэтому за основу были взяты сценарии из раздела 1.7. В Таблице 2.1 приведён реестр индексов базы данных ViewTrain.

Таблица 2.1 — Реестр индексов базы данных ViewTrain

Индекс	Таблица	Столбцы	Тип	Обоснование	Авто?
users_pkey	users	id	B-Tree	Поиск по РК	Да
users_email_key	users	email	B-Tree	Авторизация — поиск по email	Да (UNIQUE)
sessions_pkey	sessions	id	B-Tree	Поиск по РК	Да
idx_sessions_user_status	sessions	(user_id, status)	B-Tree	Список сессий — главная страница	Нет
messages_pkey	messages	id	B-Tree	Поиск по РК	Да
idx_messages_session_id	messages	(session_id, id)	B-Tree	История переписки перед каждым запросом GigaChat	Нет
idx_refresh_token_hash	refresh_tokens	token_hash	B-Tree	Обновление access-токена	Нет
refresh_tokens_pkey	refresh_tokens	id	B-Tree	Поиск по РК	Да
questions_pkey	questions	id	B-Tree	Поиск по РК	Да

В Листинге 2.6 представлен DDL-скрипт для создания индексов схемы ViewTrain.

Листинг 2.6 — DDL-скрипт создания индексов

```
-- Список сессий пользователя с фильтрацией по статусу
CREATE INDEX idx_sessions_user_status
  ON sessions (user_id, status);

-- Составной индекс по (session_id, id) даёт порядок без ORDER BY
CREATE INDEX idx_messages_session_id
  ON messages (session_id, id);

-- Поиск refresh-токена по SHA-256-хэшу
CREATE INDEX idx_refresh_token_hash
  ON refresh_tokens (token_hash);
```

Стоит подробнее объяснить, почему индекс `idx_messages_session_id` построен именно по двум полям. Запрос, который загружает историю переписки перед каждым обращением к GigaChat, делает две вещи одновременно: отбирает сообщения нужной сессии и возвращает их в хронологическом порядке. Если использовать индекс только по полю `session_id`, то он закрыл бы лишь первую часть. Строки бы нашлись, но PostgreSQL всё равно пришлось бы их отсортировать. Для того, чтобы закрыть вторую часть используется второе поле — `id`. Теперь строки внутри индекса уже упорядочены, и база отдаёт результат без каких-либо дополнительных операций.

2.6 Транзакции и обеспечение целостности данных

В некоторых сценариях недостаточно просто выполнить запрос — нужно, чтобы несколько операций либо прошли все вместе, либо не прошла ни одна.

Первый такой сценарий — завершение сессии. Когда собеседование заканчивается, нужно обновить статус сессии и записать последнее сообщение. Если сервер упадёт между этими двумя операциями, сессия зависнет в статусе «активна» без финального сообщения. Чтобы этого не случилось, обе операции выполняются в одной транзакции.

Второй сценарий — обновление refresh-токена. Здесь важно не просто выдать новый токен, но и аннулировать старый. Если между этими двумя шагами что-то пойдёт не так, пользователь останется без рабочего токена. Решение этой проблемы аналогично первому сценарию.

Третий сценарий — старт сессии. При первичном создании сессии в базе одновременно должны выполняться три операции: создать запись в таблице сессий, из каталога взять вопрос и вместе с ним создать запись в таблице сообщений, как первое сообщение интервьюера. Чтобы все три операции гарантированно выполнились или откатились вместе, достаточно выполнять их в рамках одной транзакции.

Помимо этого, важно понимать, как PostgreSQL ведёт себя при параллельной работе множества пользователей. В ViewTrain одновременно могут идти десятки сессий, и в каждой из них постоянно что-то читается и записывается. Благодаря MVCC чтение и запись не мешают друг другу: пока один пользователь загружает историю переписки, другой спокойно отправляет новое сообщение — никто никого не ждёт. Совокупность транзакционных гарантий и MVCC обеспечивает ту степень надёжности и согласованности данных, которая необходима ViewTrain как коммерческому продукту. Пользователи стартапа не должны сталкиваться с ошибками при работе системы. Подобные дефекты, как правило, подрывают доверие к сервису на ранней стадии его развития.

2.7 Процесс реализации и принятые решения

Все проектные решения, принятые в разделе 2, необходимо было перевести в работающую схему. Во время переноса часть решений корректировалась по ходу реализации, когда обнаруживались конфликты между теоретической моделью и поведением PostgreSQL на практике. В данном подразделе описан именно этот процесс — с акцентом на то, что пришлось пересмотреть и почему.

При создании схемы важно было соблюдать правильный порядок: нельзя создать таблицу, которая ссылается на другую, если та ещё не существует в базе — PostgreSQL просто выдаст ошибку. То же самое касается пользовательских типов: сначала они должны появиться в системе, и только потом их можно использовать в определениях таблиц. Поэтому весь DDL-скрипт выстраивался по одному принципу — сначала то, от чего зависят другие объекты, и лишь потом всё, что на них опирается.

Первым шагом создавались пользовательские перечисляемые типы: `interview_type`, `difficulty_level`, `session_status` и `message_role`. Это условие выполняется первым, так как поля таблиц сессий(`sessions`) и сообщений(`messages`) используют эти типы напрямую. Вторым шагом создавалась таблица пользователей(`users`), единственная таблица без внешних ключей, от которой зависят все остальные. Третьим — сессии(`sessions`) и токены(`refresh_tokens`), ссылающиеся на таблицу пользователей. Четвёртым — сообщения(`messages`), ссылающаяся на сессии. Пятым — вопросы(`questions`), не имеющая внешних ключей и не зависящая ни от одной другой таблицы. Последним шагом создавались индексы после того, как в таблицы были загружены тестовые данные. Данное решение объясняется соображением производительности: создание индекса по уже заполненной таблице в PostgreSQL выполняется за один проход через все строки, тогда как инкрементное обновление индекса при каждой вставке в ходе загрузки данных создаёт дополнительные накладные расходы. На тестовом датасете из 500 000 сообщений такой порядок ощутимо сократил время первоначального заполнения.

Изначально все таблицы должны были использовать UUID как первичный ключ. Но для `messages` пришлось отступить от этого правила в пользу BIGSERIAL. Причина простая: составной индекс по (`session_id`, `id`) использует `id` для сортировки строк внутри индекса. UUID случаен — он ничего не знает о порядке вставки. BIGSERIAL монотонно растёт, поэтому физический порядок строк в индексе совпадает с хронологическим.

PostgreSQL отдаёт историю переписки уже отсортированной, без лишних операций.

Реализация триггера счётчика сообщений

Денормализованное поле `message_count` в таблице сессий(`sessions`), обоснованное в подразделе 2.2, требует механизма автоматического обновления. Ручное обновление счётчика на уровне приложения создаёт риск рассогласования. К примеру, запрос к базе завершится с ошибкой после вставки сообщения, но до обновления счётчика, данные окажутся в несогласованном состоянии. Для устранения этого риска был реализован триггер на уровне СУБД и функция обновления счетчика, представленные в Листинге 2.7.

Листинг 2.7 — Триггерная функция и триггер обновления счётчика сообщений

```
CREATE OR REPLACE FUNCTION increment_message_count()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE sessions
    SET message_count = message_count + 1
    WHERE id = NEW.session_id;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_increment_message_count
AFTER INSERT ON messages
FOR EACH ROW
EXECUTE FUNCTION increment_message_count();
```

Триггер `trg_increment_message_count` срабатывает каждый раз, когда в таблицу `messages` добавляется новая строка, и сразу обновляет счётчик в соответствующей записи таблицы `sessions`. Поскольку всё это происходит в рамках одной транзакции, оба действия неразрывно связаны: если что-то пошло не так — откатятся оба, если всё прошло успешно — зафиксируются оба. Счётчик никогда не окажется в рассинхроне с реальным числом сообщений.

2.8 Стратегия тестирования

После реализации схемы необходимо было убедиться, что она ведёт себя корректно как с точки зрения целостности данных, так и с точки зрения производительности под нагрузкой. Тестирование проводилось на двух уровнях.

Первый уровень — функциональное тестирование целью которого было проверить, что все ограничения целостности, объявленные в DDL-скрипте, действительно работают корректно и не допускают некорректные значения в таблицу. Все тестирование проводилось вручную за счет выполнения SQL-запросов, где каждый тест представлял собой попытку вставить или изменить данные таким образом, чтобы нарушить то или иное ограничение и увидеть реакцию базы данных.

Второй уровень — нагрузочное тестирование. Его цель — проверить, что критичные запросы выполняются в пределах нефункциональных требований при реалистичном объёме данных. При таком тестировании важно было отслеживать анализ запросов. Для таких целей в PostgreSQL существует команда EXPLAIN ANALYZE, она не только показывает план выполнения запроса, но и фактически его исполняет, фиксируя реальное время каждой операции и число обработанных строк. Чтобы анализ был близок к реалистичным сценариям на проде пришлось вручную с помощью SQL-скриптов генерировать данные: 10 000 пользователей, 50 000 сессий и 500 000 сообщений.

2.9 Функциональное тестирование

В функциональное тестирование входило четыре вида ограничений целостности: UNIQUE, CASCADE DELETE, ENUM и CHECK. Для каждого из них выполнялась серия тест-кейсов; ниже приведены наиболее показательные из них.

UNIQUE

Ограничение уникальности поля email таблицы users должно отклонять попытку зарегистрировать двух пользователей с одинаковым адресом электронной почты. Для проверки в базу была вставлена запись с адресом test@viewtrain.ru, после чего выполнена повторная вставка с тем же значением поля. Соответствующие SQL-запросы представлены в Листинге 2.8.

Листинг 2.8 — Проверка UNIQUE-ограничения на поле email

```
-- Первая вставка должна пройти успешно
INSERT INTO users (email, username, hashed_password)
VALUES ('test@viewtrain.ru', 'user_one', 'hash_one');

-- Вторая вставка с тем же email должна быть отклонена
INSERT INTO users (email, username, hashed_password)
VALUES ('test@viewtrain.ru', 'user_two', 'hash_two');
```

В результате вторая вставка завершилась с ошибкой ERROR: duplicate key value violates unique constraint «users_email_key» и в таблице осталась только первая запись, что дает подтверждение корректной работы ограничения на уникальность.

CASCADE DELETE

Каскадное удаление — одно из ключевых требований схемы так как при удалении пользователя должны автоматически удаляться все его сессии и сообщения внутри них. Чтобы проверить такой тип удаления был создан пользователь, две сессии и по три сообщения в каждой. После удаления пользователя выполнялась проверка состояния таблиц сессий и сообщений. Проверка представлена в Листинге 2.9.

Листинг 2.9 — Проверка каскадного удаления

```
DELETE FROM users WHERE email = 'test@viewtrain.ru';

SELECT COUNT(*) FROM sessions WHERE user_id = '<uuid>';

SELECT COUNT(*) FROM messages
WHERE session_id IN (
    SELECT id FROM sessions WHERE user_id = '<uuid>'
);
```

В результате этого теста оба запроса вернули 0. Цепочка каскадного удаления пользователя, сессий и сообщений сработала корректно. PostgreSQL сам проходит эту цепочку и удаляет дочерние записи.

ENUM

Перечисляемый тип `interview_type` таблицы сессий допускает только три значения: `algorithms`, `system_design` и `behavioral`. Если попытаться вставить любое другое значение СУБД должна отклонить такую операцию. Проверка представлена в Листинге 2.10.

Листинг 2.10 — Проверка ENUM-ограничения

```
INSERT INTO sessions (user_id, interview_type, difficulty)
VALUES ('<uuid>', 'math', 'junior');
```

Как и ожидалось результат — ошибка `ERROR: invalid input value for enum interview_type: «math»` из которой следует, что валидация работает и гарантирует целостность данных даже при обходе логики приложения.

CHECK

Ограничение на таблице `refresh_tokens` служило проверкой, что срок истечения токена всегда позже момента его создания, но в процессе тестирования был выявлен дефект.

Первоначальный вид ограничения был как `expires_at > NOW()`. При ручном тестировании обнаружилось, что вставка корректного короткосрочного токена периодически завершалась ошибкой `CheckViolation`. При анализе была выявлена следующая причина: между моментом вычисления значения `expires_at` на стороне приложения и моментом фактического выполнения добавления в базе проходило какое-то время, за которое PostgreSQL вычислял `NOW()` заново, и в редких случаях это новое значение оказывалось позже, чем ожидалось. В Листинге 2.11 показано исправленное CHECK-ограничение.

Листинг 2.11 — Исправленное CHECK-ограничение

```
-- Было
CONSTRAINT chk_expires CHECK (expires_at > NOW())

-- Стало
CONSTRAINT chk_expires CHECK (expires_at > created_at)
```

После устранения данной проблемы ограничение стало сравнивать два этих поля одной строки в момент вставки, что исключило зависимость от системного времени и избавило от ложных срабатываний.

Итоги функционального тестирования

После того как провелись все тестирования был выявлен один дефект ограничения таблицы `refresh_tokens`, который впоследствии устранили до финальной версии схемы. По итогу конечная версия DDL скрипта прошла все проверки успешно.

2.10 Нагрузочное тестирование и оптимизация

Нагрузочное тестирование было сосредоточено на трёх запросах, которые по результатам анализа паттернов доступа в подразделе 1.7 были определены как критичные: загрузка истории переписки сессии, выборка списка сессий пользователя и поиск refresh-токена по хэшу. Для каждого из них была получена пара планов выполнения — до создания индексов и после — с помощью команды `EXPLAIN ANALYZE`.

Загрузка истории переписки сессии — это самый частый запрос системы. Он выполняется перед каждым обращением к GigaChat, то есть при каждом новом сообщении пользователя. Запрос выбирает все сообщения заданной сессии в хронологическом порядке. Сам запрос представлен в Листинге 2.12.

Листинг 2.12 — Запрос истории переписки

```
SELECT role, content
FROM messages
WHERE session_id = '<uuid сессии>'
ORDER BY id ASC;
```

До создания индекса планировщик использовал последовательное сканирование (Seq Scan), то есть он читал все строки таблицы `messages` и для каждой проверял, принадлежит ли она нужной сессии. Вывод `EXPLAIN ANALYZE` до создания индекса представлен в Листинге 2.13.

Листинг 2.13 — EXPLAIN ANALYZE до индекса

```
Seq Scan on messages  (cost=0.00..12480.00 rows=50 width=120)
    (actual time=0.043..312.412 rows=50 loops=1)
    Filter: (session_id = '...':::uuid)
    Rows Removed by Filter: 499950
Execution Time: 312.5 ms
```

Из 500 000 строк таблицы планировщик вынужден был проверить 499 950 и отбросить их как нерелевантные. После создания составного индекса `idx_messages_session_id` по полям (`session_id`, `id`) планировщик переключился на `Index Scan`. Вывод `EXPLAIN ANALYZE` после создания индекса представлен в Листинге 2.14.

Листинг 2.14 — EXPLAIN ANALYZE после индекса idx_messages_session_id

```
Index Scan using idx_messages_session_id on messages
(cost=0.42..8.44 rows=50 width=120)
(actual time=0.021..8.034 rows=50 loops=1)
Index Cond: (session_id = '... '::uuid)
Execution Time: 8.1 ms
```

Время выполнения сократилось с 312 мс до 8 мс — ускорение в 39 раз. Строки возвращаются уже упорядоченными по полю `id`, поскольку второй столбец составного индекса хранит их именно в таком порядке: отдельная операция сортировки отсутствует в плане выполнения.

Выборка списка сессий пользователя — данный запрос выполняется при загрузке страницы пользователя и личного кабинета. Он возвращает сессии конкретного пользователя, отфильтрованные по статусу. Запрос представлен в Листинге 2.15.

Листинг 2.15 — Запрос списка сессий пользователя

```
SELECT id, interview_type, difficulty, status, created_at, message_count
FROM sessions
WHERE user_id = '<uuid пользователя>'
AND status = 'completed'
ORDER BY created_at DESC;
```

Вывод `EXPLAIN ANALYZE` до создания индекса представлен в Листинге 2.16.

Листинг 2.16 — EXPLAIN ANALYZE до индекса

```
Seq Scan on sessions (cost=0.00..1250.00 rows=10 width=80)
(actual time=0.031..187.340 rows=10 loops=1)
Filter: ((user_id = '... '::uuid) AND (status =
'completed'::session_status))
Rows Removed by Filter: 49990
Execution Time: 187.4 ms
```

Вывод `EXPLAIN ANALYZE` после создания индекса `idx_sessions_user_status` представлен в Листинге 2.17.

Листинг 2.17 — EXPLAIN ANALYZE после индекса idx_sessions_user_status

```
Index Scan using idx_sessions_user_status on sessions
```

```
(cost=0.29..4.12 rows=10 width=80)
(actual time=0.018..12.230 rows=10 loops=1)
Index Cond: ((user_id = '... '::uuid) AND
              (status = 'completed'::session_status))
Execution Time: 12.3 ms
```

Ускорение составило 16 раз: с 187 мс до 12 мс. Составной индекс по (user_id, status) закрывает оба условия фильтрации одновременно — планировщик не сканирует строки других пользователей и не проверяет строки с другим статусом.

Поиск токена обновления по хэшу — этот запрос происходит каждый раз, когда пользователю нужно получить новый access-токен. Приложение отправляет в базу SHA-256-хэш токена, и та должна оперативно найти нужную запись. Запрос представлен в Листинге 2.18.

Листинг 2.18 — Запрос поиска токена по хэшу

```
SELECT id, user_id, expires_at, revoked
FROM refresh_tokens
WHERE token_hash = '<sha256-хэш>';
```

Вывод EXPLAIN ANALYZE до создания индекса представлен в Листинге 2.19.

Листинг 2.19 — EXPLAIN ANALYZE до индекса

```
Seq Scan on refresh_tokens (cost=0.00..680.00 rows=1 width=60)
      (actual time=0.028..143.210 rows=1 loops=1)
  Filter: (token_hash = '... '::varchar)
  Rows Removed by Filter: 29999
Execution Time: 143.2 ms
```

Вывод EXPLAIN ANALYZE после создания индекса idx_refresh_token_hash представлен в Листинге 2.20.

Листинг 2.20 — EXPLAIN ANALYZE после индекса idx_refresh_token_hash

```
Index Scan using idx_refresh_token_hash on refresh_tokens
(cost=0.29..2.51 rows=1 width=60)
(actual time=0.014..3.120 rows=1 loops=1)
Index Cond: (token_hash = '... '::varchar)
Execution Time: 3.1 ms
```

Ускорение составило 48 раз с 143 мс до 3 мс. Поле token_hash обладает максимальной селективностью, поэтому индекс позволяет найти нужную строку практически мгновенно.

Влияние индексов на операции записи

Создание индексов ускоряет чтение, но создаёт накладные расходы на запись. При каждой вставке PostgreSQL обязан обновить все индексы на изменённой таблице. Для таблицы `messages`, где сосредоточена основная нагрузка по записи, этот эффект был измерен отдельно. Результаты измерений представлены в Таблице 2.2.

Таблица 2.2 — Влияние индекса на время `INSERT` в таблицу `messages`

Условие	Среднее время вставки (мс)
Без индекса <code>idx_messages_session_id</code>	7
С индексом <code>idx_messages_session_id</code>	9

Рост составил 28%: с 7 мс до 9 мс. Оба эти значения укладываются в нефункциональное требование НФТ-БД-02, поэтому компромисс признан приемлемым. В системе, где история загружается при каждом новом сообщении, а запись происходит лишь единожды, замедление вставки на 28% полностью оправдывается ускорением чтения в 39 раз.

Сводные результаты нагрузочного тестирования

Сводные результаты нагрузочного тестирования представлены в Таблице 2.3.

Таблица 2.3 — Результаты нагрузочного тестирования

Запрос	Без индекса (мс)	С индексом (мс)	Ускорение	Выполнение требования
История сессии (50 сообщений)	312	8	39×	НФТ-БД-01: цель не более 50 мс ✓
Список сессий пользователя	187	12	16×	НФТ-БД-03: цель не более 200 мс ✓
Поиск <code>refresh</code> -токена по хэшу	143	3	48×	НФТ-БД-04: цель не более 50 мс ✓
<code>INSERT</code> нового сообщения	7	9	−0,3×	НФТ-БД-02: цель не более 10 мс ✓

Все нефункциональные требования по производительности выполнены.

Это означает, что спроектированная база данных готова обеспечивать работу ViewTrain при реалистичной нагрузке стартапа на стадии активного

роста. Также она не создаст технических ограничений для дальнейшего масштабирования сервиса по мере увеличения пользовательской базы.

3 ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ ПРОЕКТА

3.1 Затраты на разработку

При подсчетах затрат на разработку базы данных для веб-приложения ViewTrain учитывалась следующая трудоемкость по задачам: проектирование моделей — 40 часов, написание DDL-скрипта и триггеров — 20 часов, разработка стратегии индексирования — 15 часов, функциональное тестирование — 35 часов, нагрузочное тестирование и оптимизация — 50 часов. Если взять в расчет среднюю часовую ставку для средних специалистов по Москве в 2 500 рублей, то общая стоимость работ выходит в 400 000 тысяч. Сводные данные по затратам на разработку приведены в Таблице 3.1.

Таблица 3.1 — Условные затраты на разработку базы данных

Этап работ	Часов	Стоимость, Р
Проектирование моделей данных	40	100 000
Реализация DDL и триггеров	20	50 000
Стратегия индексирования	15	37 500
Функциональное тестирование	35	87 500
Нагрузочное тестирование и оптимизация	50	125 000
Итого	160	400 000

3.2 Затраты на эксплуатацию

Как уже говорилось ранее, PostgreSQL не подразумевает вложений при использовании, однако, без аренды сервера, на котором всё будет работать не обойтись. Если взять в расчет 1 000 активных пользователей, то под такую нагрузку подойдет конфигурация на 4 ГБ оперативной памяти и 50 ГБ дискового пространства. Стоимость такой конфигурации варьируется от 3 000

до 5 000, а если взять в расчет резервное копирование и мониторинг, то конечная сумма составит около 6 000 тысяч в месяц или 72 000 тысяч в год.

3.3 Оценка эффективности проектных решений

При оценке эффективных проектных решений основной акцент делается на экономии на инфраструктуре и удержание целевой аудитории. На начальном этапе важно реализовать такую систему, которая будет изначально выгодна в содержании. Для ситуации, когда база не оптимизирована, потребовался бы более дорогостоящий сервер, который оценивается около 20 000 рублей в месяц. Благодаря оптимизации во время разработки получилось сократить время ожидания ответа в несколько десятков раз. За счет такого подхода требование в мощном сервере отпало, что позволило сэкономить 120 000–140 000 рублей. Не менее важный аспект — удержание пользователей. Если сервис будет медленным, то есть риск, что это может повлечь за собой отток пользователей. За счет оптимизации и проработанной системы, есть вероятность, что пользователей задержится, что за собой повлечет уменьшение расходов на повторное привлечение. По оценкам российских EdTech-компаний, стоимость привлечения одного пользователя составляет от 300 до 800 рублей. Даже при удержании 5% пользователей из ежемесячного притока в 500 человек — экономия около 12 500 рублей в месяц или 150 000 рублей в год.

3.4 Окупаемость

Если в расчет взять стоимость подписки 499–799 рублей в месяц и конверсию бесплатных пользователей в платных на уровне 3–5%, то точка безубыточности достигается при 200–300 платящих пользователях. С учетом возможных дополнительных трат ожидается выход на прибыльность уже через 12–18 месяцев. Сводные показатели экономической оценки приведены в Таблице 3.2.

Таблица 3.2 — Сводные показатели экономической оценки

Показатель	Значение
Условная стоимость разработки БД	400 000 Р
Затраты на инфраструктуру в год (старт)	72 000 Р
Экономия на инфраструктуре при росте	120 000–140 000 Р в год
Экономия на удержании пользователей	≈ 150 000 Р в год
Точка безубыточности	200–300 платящих подписчиков
Срок выхода на прибыльность	12–18 месяцев

В итоге база данных решает не только технические задачи, но и помогает проекту экономить, а также удерживать пользователей. Для стартапа, который только начинает свое развитие на рынке это очень важно.

ЗАКЛЮЧЕНИЕ

В ходе выпускной квалификационной работы была спроектирована и реализована реляционная база данных для веб-приложения ViewTrain — технологического стартапа в сфере EdTech, представляющего собой платформу для подготовки к техническим собеседованиям с AI-интервьюером на базе GigaChat. Все задачи, поставленные во введении, выполнены.

В первой главе проведён анализ предметной области методом query-driven design и были выявлены пять основных сущностей системы, а также установлены критичные паттерны доступа к данным. Параллельно был выполнен анализ существующих платформ для подготовки к собеседованиям, который показал, что прямые аналоги ViewTrain на рынке отсутствуют, а ориентация на русскоязычную аудиторию и использование отечественной языковой модели GigaChat формируют основу рыночного позиционирования стартапа. Также была проработана бизнес-модель проекта, основой которой послужил freemium-подход. Вдобавок к этому были определены направления потенциального масштабирования — от расширения тематики интервью до выхода в B2B-сегмент и на международный рынок. Из трёх рассмотренных систем — PostgreSQL, MongoDB и Redis, был выбран PostgreSQL. Причина такого выбора — только он из всех кандидатов даёт полные гарантии целостности и надёжности данных, нормально работает с параллельными запросами через MVCC и умеет сложные выборки с агрегациями. Дополнительным аргументом в пользу PostgreSQL стало отсутствие лицензионных отчислений, что для стартапа на ранней стадии означает минимизацию инфраструктурных затрат. Чтобы было понятно, что именно должна уметь база данных и насколько хорошо, были сформулированы шесть функциональных и четыре нефункциональных требований — каждое с конкретной цифрой или условием.

Во второй главе данные были описаны на трёх уровнях. Сначала концептуальная модель. Затем логическая модель с нормализацией до третьей

нормальной формы. И наконец физическая схема, где были реализованы готовые DDL-скрипты с пользовательскими типами, ограничениями и каскадным удалением. Отдельно была продумана стратегия индексирования — три B-Tree индекса, каждый из которых заточен под конкретный сценарий работы серверной части. В этой же главе был описан процесс реализации схемы и зафиксировано одно решение, скорректированное по ходу работы. Также было проведено функциональное тестирование, где был выявлен один дефект, связанный с CHECK-ограничением таблицы `refresh_tokens`, который устранили до финальной версии схемы. Нагрузочное же тестирование прошло полностью успешно и на датасете из 500 000 сообщений подтвердило выполнение всех нефункциональных требований.

В третьей главе было выполнено экономическое обоснование проекта. Условные затраты на разработку базы данных оценены в 400 000 рублей, а эксплуатационные расходы на старте — около 72 000 рублей в год. Ожидаемая экономия на серверной инфраструктуре по мере роста пользовательской базы составляет порядка 120–140 тысяч рублей в год, а косвенный эффект от удержания пользователей за счёт быстрого отклика приложения — ещё около 150 тысяч рублей в год. Чтобы проект вышел в ноль, нужно набрать 200–300 платящих пользователей, а на стабильную прибыль он должен выйти примерно через год-полтора после запуска.

Спроектированная база данных уже используется серверной частью ViewTrain и готова к реальной эксплуатации в составе действующего стартап-проекта. Схема с самого начала проектировалась не абстрактно, а под конкретные запросы серверной части — поэтому никаких расхождений с бэкендом не возникло.

Реализованная база данных уже сейчас обеспечивает запас прочности под рост пользовательской базы и не потребует архитектурных переработок на ближайшем этапе масштабирования сервиса. А продуманная бизнес-модель и обозначенные направления развития создают основу для дальнейшей коммерциализации ViewTrain как полноценного стартап-продукта.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Клеппман М. Высоконагруженные приложения. Программирование, масштабирование, поддержка / Пер. с англ. — СПб.: Питер, 2018. — 640 с.
2. Тихомиров В.С. Базы данных и SQL. Практический курс. — СПб.: БХВ-Петербург, 2023. — 448 с.
3. Фёдоров А.В. PostgreSQL: профессиональное применение. — СПб.: БХВ-Петербург, 2022. — 496 с.
4. Кузнецов О.А. Проектирование реляционных баз данных. — М.: ДМК Пресс, 2021. — 368 с.
5. Павлов А.Б. Тестирование программного обеспечения. — М.: ДМК Пресс, 2022. — 256 с.
6. Сергеев К.И. Оптимизация запросов в реляционных СУБД. — М.: Питер, 2023. — 320 с.
7. Kleppmann M. Designing Data-Intensive Applications. — O'Reilly Media, 2017. — 616 p.
8. Elmasri R., Navathe S. Fundamentals of Database Systems. 7th ed. — Pearson, 2015. — 1280 p.
9. Karwin B. SQL Antipatterns. — Pragmatic Bookshelf, 2010. — 328 p.
10. Winand M. SQL Performance Explained. — Winand, 2012. — 196 p.
11. Reti D. PostgreSQL Query Optimization. — Apress, 2022. — 344 p.
12. Date C.J. An Introduction to Database Systems. 8th ed. — Addison-Wesley, 2003. — 1024 p.
13. Momjian B. PostgreSQL: Introduction and Concepts. — Addison-Wesley, 2001. — 461 p.
14. PostgreSQL 15 Documentation. — Режим доступа: <https://www.postgresql.org/docs/15/> (дата обращения: 10.02.2026).

15. Use The Index, Luke! — руководство по производительности БД. — Режим доступа: <https://use-the-index-luke.com> (дата обращения: 15.11.2025).
16. GigaChat API. Документация разработчика. — Режим доступа: <https://developers.sber.ru/docs/ru/gigachat/overview> (дата обращения: 13.01.2026).
17. Кириллов В.В., Громов Г.Ю. Введение в реляционные базы данных. — СПб.: БХВ-Петербург, 2012. — 464 с.
18. Ports D., Grittner K. Serializable Snapshot Isolation in PostgreSQL // Proceedings of the VLDB Endowment. — 2012. — Vol. 5, No. 12. — P. 1850–1861.

ПРИЛОЖЕНИЯ

Приложение А

Полный DDL-скрипт схемы базы данных ViewTrain

В данном приложении представлен полный SQL-скрипт создания схемы.

В Листинге А.1 представлен полный DDL-скрипт ViewTrain.

Листинг А.1 — Полный DDL-скрипт ViewTrain

```
CREATE TYPE interview_type AS ENUM
    ('algorithms', 'system_design', 'behavioral');

CREATE TYPE difficulty_level AS ENUM ('junior', 'middle', 'senior');

CREATE TYPE session_status AS ENUM
    ('created', 'active', 'completed', 'abandoned');

CREATE TYPE message_role AS ENUM ('user', 'assistant');

CREATE TABLE users (
    id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
    email             VARCHAR(255)        UNIQUE NOT NULL,
    username          VARCHAR(100)        NOT NULL,
    hashed_password   VARCHAR(255)        NOT NULL,
    created_at        TIMESTAMPTZ         NOT NULL DEFAULT NOW(),
    is_active         BOOLEAN             NOT NULL DEFAULT TRUE
);

CREATE TABLE sessions (
    id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id           UUID                NOT NULL
        REFERENCES users(id) ON DELETE CASCADE,
    interview_type    interview_type      NOT NULL,
    difficulty         difficulty_level    NOT NULL,
    status            session_status       NOT NULL DEFAULT 'created',
    message_count     INTEGER              NOT NULL DEFAULT 0,
    started_at        TIMESTAMPTZ,
    ended_at          TIMESTAMPTZ,
    created_at        TIMESTAMPTZ         NOT NULL DEFAULT NOW()
);

CREATE TABLE refresh_tokens (
    id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id           UUID                NOT NULL
        REFERENCES users(id) ON DELETE CASCADE,
    token_hash        VARCHAR(64)         UNIQUE NOT NULL,
    expires_at        TIMESTAMPTZ         NOT NULL,
    revoked           BOOLEAN              NOT NULL DEFAULT FALSE,
    created_at        TIMESTAMPTZ         NOT NULL DEFAULT NOW(),
    CONSTRAINT chk_expires CHECK (expires_at > created_at)
);

CREATE TABLE messages (
```



```

        id          BIGSERIAL      PRIMARY KEY,
        session_id  UUID           NOT NULL
                        REFERENCES sessions(id) ON DELETE CASCADE,
        role        message_role NOT NULL,
        content     TEXT           NOT NULL,
        created_at  TIMESTAMPTZ    NOT NULL DEFAULT NOW()
    );

CREATE TABLE questions (
    id          UUID              PRIMARY KEY DEFAULT gen_random_uuid(),
    interview_type interview_type NOT NULL,
    difficulty   difficulty_level NOT NULL,
    content     TEXT             NOT NULL,
    tags        TEXT[]
);

CREATE INDEX idx_sessions_user_status
    ON sessions (user_id, status);

CREATE INDEX idx_messages_session_id
    ON messages (session_id, id);

CREATE INDEX idx_refresh_token_hash
    ON refresh_tokens (token_hash);

CREATE OR REPLACE FUNCTION increment_message_count()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE sessions
    SET message_count = message_count + 1
    WHERE id = NEW.session_id;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_increment_message_count
AFTER INSERT ON messages FOR EACH ROW
EXECUTE FUNCTION increment_message_count();

```

Скрипт содержит полное описание схемы. Перечисляемые типы, все пять таблиц с ограничениями, индексы и триггер обновления счётчика сообщений.